

Lecture 14: Introduction to the Conjugate Direction Method

Instructor Proof Notes

Color Legend

Green: Step numbering and substeps **Orange:** Key concepts **Purple:** Main results **Blue:** Teaching notes

Proof Roadmap

Theorem	Time	Board-Friendly	Priority
Theorem 1: Q-Conjugate Directions $\Rightarrow$ Linear Independence	5–8 min	Yes	Full
Theorem 2: Exact Step Size Formula	5–8 min	Yes	Full
Theorem 3: Gradient Orthogonality Property	10–12 min	Yes	Full
Theorem 4: Expanding Subspace Theorem	12–15 min	Moderate	Full

1 Theorem 1: Q-Conjugate Directions Are Linearly Independent

Statement: Let Q be a symmetric positive definite (SPD) matrix. If $d_1, d_2, \dots, d_k \in \mathbb{R}^n$ are **Q-conjugate directions**, meaning $d_i^T Q d_j = 0$ for all $i \neq j$, then they are **linearly independent**.

Proof:

Step 1: Suppose the directions are linearly dependent. Then there exist scalars $\alpha_1, \alpha_2, \dots, \alpha_k$ (not all zero) such that

$$\alpha_1 d_1 + \alpha_2 d_2 + \dots + \alpha_k d_k = 0.$$

Step 2: Multiply both sides on the left by $d_i^T Q$ for any fixed $i \in \{1, \dots, k\}$:

$$d_i^T Q (\alpha_1 d_1 + \alpha_2 d_2 + \dots + \alpha_k d_k) = d_i^T Q \cdot 0 = 0.$$

Step 3: Expand the left side:

$$\alpha_1 (d_i^T Q d_1) + \alpha_2 (d_i^T Q d_2) + \dots + \alpha_i (d_i^T Q d_i) + \dots + \alpha_k (d_i^T Q d_k) = 0.$$

Step 4: By Q-conjugacy, $d_i^T Q d_j = 0$ for all $j \neq i$. Thus all cross terms vanish, leaving

$$\alpha_i (d_i^T Q d_i) = 0.$$

Step 5: Since Q is SPD and $d_i \neq 0$ (all directions are nonzero), we have $d_i^T Q d_i > 0$. Therefore,

$$\alpha_i = 0.$$

Step 6: Since i was arbitrary, $\alpha_1 = \alpha_2 = \dots = \alpha_k = 0$, contradicting the assumption of linear dependence.

The directions $d_1, \dots, d_k$ are linearly independent.

Teaching Notes: Theorem 1

Board Strategy: This is a short, clean proof — excellent for chalk talk. Draw an analogy with ordinary orthogonality first.

Key Insight: Q -conjugacy is the “ Q -inner product” analogy to orthogonality. Just as orthogonal vectors are linearly independent, so are Q -conjugate vectors.

Diagram: Draw 2D example: two orthogonal vectors v_1, v_2 are linearly independent. Then explain that Q -conjugacy replaces the standard dot product with the Q -weighted inner product $d_i^T Q d_j$.

Motivation: Emphasize that we need conjugacy (not just any directions) to guarantee independence. This is foundational to why the algorithm works.

Timing: 5–8 minutes, including the analogy. This proof can be rushed if time is tight.

Common Student Confusion

Q: “Why can’t we just use orthogonal directions instead of Q -conjugate?”

A: Because the underlying optimization problem is weighted by Q . The function $f(x) = \frac{1}{2}x^T Q x + c^T x$ is naturally expressed in the Q -inner product. Directions that are orthogonal in the standard sense may not be “aligned” with the contours of f . Q -conjugacy ensures that once we optimize along d_i , we never need to revisit that direction again.

Analogy: Think of Q as defining an elliptical metric. In this metric, conjugate vectors are truly “separated” and don’t interfere with each other. Standard orthogonal vectors in $\mathbb{R}^n$ would have overlapping components when viewed through the Q -metric.

2 Theorem 2: Exact Step Size Formula

Statement: Consider minimizing $f(x) = \frac{1}{2}x^T Q x + c^T x$ (with Q SPD) along a direction d_k starting from x_k . The **exact line search** that minimizes $f(x_k + \lambda d_k)$ over $\lambda \in \mathbb{R}$ gives the step size

$$\lambda_k = -\frac{\nabla_k^T d_k}{d_k^T Q d_k},$$

where $\nabla_k = \nabla f(x_k) = Q x_k + c$.

Derivation:

Step 1: Define the univariate function

$$\phi(\lambda) = f(x_k + \lambda d_k) = \frac{1}{2}(x_k + \lambda d_k)^T Q (x_k + \lambda d_k) + c^T (x_k + \lambda d_k).$$

Step 2: Expand:

$$\phi(\lambda) = \frac{1}{2} (x_k^T Q x_k + 2\lambda x_k^T Q d_k + \lambda^2 d_k^T Q d_k) + c^T x_k + \lambda c^T d_k \quad (1)$$

$$= \frac{1}{2} x_k^T Q x_k + c^T x_k + \lambda (x_k^T Q d_k + c^T d_k) + \frac{\lambda^2}{2} d_k^T Q d_k. \quad (2)$$

Step 3: Recognize that $x_k^T Q d_k + c^T d_k = (Q x_k + c)^T d_k = \nabla_k^T d_k$:

$$\phi(\lambda) = f(x_k) + \lambda(\nabla_k^T d_k) + \frac{\lambda^2}{2} d_k^T Q d_k.$$

Step 4: To minimize, take the derivative and set it to zero:

$$\frac{d\phi}{d\lambda} = \nabla_k^T d_k + \lambda d_k^T Q d_k = 0.$$

Step 5: Solve for λ_k :

$$\lambda_k = -\frac{\nabla_k^T d_k}{d_k^T Q d_k}.$$

Step 6: Verify this is a minimizer: Since $d_k^T Q d_k > 0$ (by positive definiteness of Q and $d_k \neq 0$), we have $\frac{d^2\phi}{d\lambda^2} = d_k^T Q d_k > 0$, confirming a minimum.

The exact step size is $\lambda_k = -\frac{\nabla_k^T d_k}{d_k^T Q d_k}$.

Teaching Notes: Theorem 2

Board Strategy: This derivation is fundamental and builds directly on univariate calculus. Write it step by step on the board. Students will see it's essentially completing the square for a quadratic in λ .

Key Insight: The numerator $-\nabla_k^T d_k$ captures how aligned the gradient is with the descent direction. The denominator $d_k^T Q d_k$ captures the curvature. Together, they give the optimal distance to travel.

Comparison: Compare to gradient descent: for steepest descent ($d_k = -\nabla_k$), we get $\lambda_k = \frac{\|\nabla_k\|^2}{\nabla_k^T Q \nabla_k}$, which is more complex due to the coupling with the step direction.

Numerator Sign: Note that $-\nabla_k^T d_k > 0$ when d is a descent direction. For conjugate directions we'll construct, this is always true.

Timing: 5–8 minutes. This is a quick, high-confidence proof that students appreciate.

Common Student Confusion

Q: “Why do we need this exact formula? Can't we just use the gradient descent step size?”

A: The exact step size is what allows the conjugate direction method to have its magical property: after stepping along d_k , the gradient becomes orthogonal to d_k (and all previous directions). Approximate line searches break this property and destroy finite termination. The exact formula is essential.

Note: For non-quadratic functions, this formula does not apply, and we must use approximate line searches (e.g., Armijo, Wolfe). This is why the finite-termination guarantee is lost for general nonlinear optimization.

3 Theorem 3: Gradient Orthogonality Property

Statement: In the **conjugate direction method**, suppose we minimize $f(x) = \frac{1}{2}x^T Qx + c^T x$ starting from x_1 with directions $d_1, \dots, d_n$ that are Q-conjugate. At iteration k , denote $\nabla_k = \nabla f(x_k)$. Then,

$$\nabla_{k+1}^T d_i = 0 \quad \text{for all } i = 1, 2, \dots, k.$$

Proof (by induction on k):

Base case ($k = 1$):

Step 1: At iteration 1, we compute $x_2 = x_1 + \lambda_1 d_1$ where λ_1 minimizes $f(x_1 + \lambda d_1)$ (exact line search).

Step 2: By first-order optimality of the line search,

$$\left. \frac{d}{d\lambda} f(x_1 + \lambda d_1) \right|_{\lambda=\lambda_1} = 0.$$

Step 3: Compute the derivative:

$$\frac{d}{d\lambda} f(x_1 + \lambda d_1) = \nabla f(x_1 + \lambda d_1)^T d_1 = (Q(x_1 + \lambda d_1) + c)^T d_1.$$

Step 4: At $\lambda = \lambda_1$, this is $\nabla_2^T d_1 = 0$. ✓

Inductive step: Assume $\nabla_k^T d_i = 0$ for all $i = 1, \dots, k-1$. Show $\nabla_{k+1}^T d_i = 0$ for all $i = 1, \dots, k$.

Step 5: For $i = k$, the base case argument applies: $\nabla_{k+1}^T d_k = 0$ from the line search at iteration k .

Step 6: For $i < k$, we write out ∇_{k+1} in terms of ∇_k :

$$\nabla_{k+1}^T d_i = (Qx_{k+1} + c)^T d_i \tag{3}$$

$$= (Q(x_k + \lambda_k d_k) + c)^T d_i \tag{4}$$

$$= (Qx_k + c)^T d_i + \lambda_k (Qd_k)^T d_i \tag{5}$$

$$= \nabla_k^T d_i + \lambda_k d_k^T Q d_i. \tag{6}$$

Step 7: By the induction hypothesis, $\nabla_k^T d_i = 0$ for $i < k$. By Q-conjugacy (since $i < k$), we have $d_k^T Q d_i = 0$. Thus both terms on the right vanish:

$$\nabla_{k+1}^T d_i = 0 + 0 = 0. \quad \checkmark$$

By induction, $\nabla_{k+1}^T d_i = 0$ for all $i = 1, \dots, k$.

Teaching Notes: Theorem 3

Board Strategy: Write the induction clearly; this is the backbone of the whole theory. Use two colors: one for the induction hypothesis, one for the Q-conjugacy property. Emphasize that **both** are needed.

Key Insight: The magic is in Step 7: two terms cancel separately. The first cancellation uses the induction hypothesis ($\nabla_k^T d_i = 0$), which says the gradient has no component along past directions. The second cancellation uses Q-conjugacy ($d_k^T Q d_i = 0$ for $i < k$), which says the new step direction is orthogonal (in the Q-sense) to all past directions. Together, these ensure the new gradient is also orthogonal to past directions.

Intuition: After we move along d_k , the gradient is orthogonal (in the Q-sense) to all previous directions. We never need to revisit them. This is why conjugate direction methods are so efficient: each iteration expands into genuinely new territory.

Visualization: In 2D or 3D, draw the iterates x_1, x_2, x_3 and the gradients $\nabla_1, \nabla_2, \nabla_3$. Show that $\nabla_2 \perp Q d_1$ and $\nabla_3 \perp_Q (d_1, d_2)$. Use the Q-metric to visualize this orthogonality.

Emphasis: Stress that this result holds *exactly* in the quadratic case with exact line search. It's the foundation for finite termination.

Timing: 10–12 minutes, including setup and visualization. This proof is worth the time—it justifies the whole algorithm. Do not skip.

Common Student Confusion

Q: “I thought the gradient should go to zero at the minimum. How can $\nabla_{k+1}^T d_i = 0$ before we're done?”

A: The gradient ∇_{k+1} is not zero; rather, it is orthogonal (in the Q-sense) to the subspace spanned by $d_1, \dots, d_k$. The gradient still has a component in the direction d_{k+1} (and beyond, if $k < n$). Once we include all n conjugate directions and minimize in each, the gradient will have nowhere else to point, and it becomes zero.

Analogy: Think of a 3D room. After the first iteration, ∇_2 has no component along d_1 (it's in the plane perpendicular to d_1). After the second, it has no component along d_1 or d_2 (it's along the line perpendicular to both). After the third, there's nowhere left to point, so $\nabla_3 = 0$.

Q-sense clarification: By “orthogonal in the Q-sense,” we mean $\nabla_{k+1}^T d_i = 0$, not $\nabla_{k+1} \cdot d_i = 0$. These are different when $Q \neq I$. The Q-sense is the right notion for this problem.

4 Theorem 4: Expanding Subspace Theorem

Statement: After k iterations of the conjugate direction method, the current point x_{k+1} is the **global minimizer of f over the affine subspace** $\{x_1\} + \text{span}(d_1, \dots, d_k)$.

Proof:

Step 1: Define the **reduced problem**: for a displacement vector $\alpha \in \mathbb{R}^k$, let

$$\theta_k(\alpha) = f(x_1 + D^k \alpha),$$

where $D^k = [d_1 \ d_2 \ \dots \ d_k] \in \mathbb{R}^{n \times k}$ is the matrix with columns $d_1, \dots, d_k$.

Step 2: Compute the gradient of θ_k :

$$\nabla \theta_k(\alpha) = (D^k)^T \nabla f(x_1 + D^k \alpha) = (D^k)^T \nabla (x_1 + D^k \alpha).$$

Step 3: At $\alpha = \lambda^k := [\lambda_1, \lambda_2, \dots, \lambda_k]^T$ (the step sizes taken through iteration k), we have $x_1 + D^k \lambda^k = x_{k+1}$. Thus,

$$\nabla \theta_k(\lambda^k) = (D^k)^T \nabla_{k+1}.$$

Step 4: By Theorem 3, for each $i = 1, \dots, k$, we have $\nabla_{k+1}^T d_i = 0$. Therefore,

$$\nabla \theta_k(\lambda^k) = (D^k)^T \nabla_{k+1} = \begin{pmatrix} d_1^T \nabla_{k+1} \\ \vdots \\ d_k^T \nabla_{k+1} \end{pmatrix} = \begin{pmatrix} 0 \\ \vdots \\ 0 \end{pmatrix} = \mathbf{0}.$$

Step 5: The Hessian of θ_k is $(D^k)^T Q D^k$, which is SPD because D^k has full column rank (Theorem 1 ensures the columns are linearly independent) and Q is SPD.

Step 6: Since θ_k is a convex quadratic function with SPD Hessian, λ^k is the **unique global minimizer** of θ_k (by first-order and second-order optimality conditions).

The point x_{k+1} globally minimizes f over $\{x_1\} + \text{span}(d_1, \dots, d_k)$.

Teaching Notes: Theorem 4

Board Strategy: The reduced-problem view is abstract but powerful. Sketch a picture: Start at x_1 . After 1 iteration, we've minimized along the line through d_1 . After 2 iterations, we've minimized over the plane spanned by d_1, d_2 . And so on. This is the geometric heart of the algorithm.

Key Insight: This theorem is the geometric foundation. Each iteration expands the subspace over which we've optimized, and we never backtrack. By the time we've used all n conjugate directions, we've optimized over all of $\mathbb{R}^n$.

Reduced Problem Intuition: The map $\alpha \mapsto x_1 + D^k \alpha$ parameterizes all points in the subspace. The reduced problem $\theta_k(\alpha)$ is the restriction of f to this subspace. By showing $\nabla \theta_k(\lambda^k) = \mathbf{0}$ and the Hessian is SPD, we confirm that λ^k is the unique minimizer of f over the subspace.

SPD Hessian: Emphasize that $(D^k)^T Q D^k$ is SPD because (a) the columns of D^k are linearly independent (from Theorem 1) and (b) Q is SPD. This ensures a unique minimizer.

Progressive Expansion: Note that the subspaces are nested: $\{x_1\} + \text{span}(d_1) \subseteq \{x_1\} + \text{span}(d_1, d_2) \subseteq \dots \subseteq \mathbb{R}^n$. By Theorem 4, f values are monotonically decreasing.

Timing: 12–15 minutes. This is moderately complex; don't rush. The reduced-problem formulation is worth dwelling on. It's powerful and will reappear in optimization theory.

Common Student Confusion

Q: “How do we know that minimizing over $\{x_1\} + \text{span}(d_1, \dots, d_k)$ is progress toward the global minimum?”

A: Because $\{x_1\} + \text{span}(d_1, \dots, d_k)$ grows with k . After n iterations, this subspace is all of $\mathbb{R}^n$ (since $d_1, \dots, d_n$ are linearly independent and span $\mathbb{R}^n$). So we've minimized over everything. Each iteration provably reduces f (because we're moving downhill), and we're guaranteed to reach the true optimum.

Convergence Guarantee: This is the power of finite-dimension optimization. With n conjugate directions, we explore all degrees of freedom, and optimality is guaranteed.

Compare to Gradient Descent: Gradient descent also produces a decreasing sequence, but it doesn't “expand” to new subspaces cleanly. It can revisit old directions. Conjugate directions guarantee we never revisit.

5 Corollary: Convergence in n Steps

Statement: The conjugate direction method converges to the **global minimizer** of $f(x) = \frac{1}{2}x^T Qx + c^T x$ (with Q SPD) in **at most n steps**.

Proof:

Step 1: By Theorem 1, the Q -conjugate directions $d_1, \dots, d_n$ are linearly independent and thus span $\mathbb{R}^n$.

Step 2: By Theorem 4, after k iterations, x_{k+1} minimizes f over $\{x_1\} + \text{span}(d_1, \dots, d_k)$.

Step 3: When $k = n$, the subspace $\{x_1\} + \text{span}(d_1, \dots, d_n) = \mathbb{R}^n$ (the entire space).

Step 4: Therefore, x_{n+1} minimizes f over all of $\mathbb{R}^n$.

Step 5: Since f is strictly convex (Hessian Q is SPD), this global minimum is unique and satisfies $\nabla f(x_{n+1}) = 0$.

The conjugate direction method converges to the global minimizer in at most n iterations.

Teaching Notes: Corollary

Board Strategy: This is the payoff theorem—students have been waiting for it. Keep it snappy (3–5 min); it’s a simple corollary of Theorems 1–4.

Key Insight: Conjugate Direction methods are *finite-terminating*. Unlike gradient descent (which can take infinitely many steps), we are guaranteed to solve the problem in exactly n iterations (in exact arithmetic). This is a landmark result.

Caveat: This finite-termination property holds *exactly* only for quadratic functions and with exact line searches. For general nonlinear functions, we lose the finite-termination guarantee. Conjugate gradient becomes an iterative method with good (but not guaranteed finite) convergence.

Practical Impact: For a problem in 100 dimensions, CG will converge in 100 steps (in exact arithmetic). Gradient descent might take thousands. This is why CG is a gold standard for large sparse systems.

Timing: 3–5 minutes. Can be delivered quickly after Theorems 1–4. This is a victory lap.

Common Student Confusion

Q: “Does this mean conjugate direction method is better than gradient descent?”

A: For *quadratic* functions, yes—finite termination is definitive. For general nonlinear functions, both are iterative; CG often converges much faster in practice due to better direction choices, but we lose the finite-termination guarantee. The quadratic case is where conjugate directions shine theoretically. In practice, for large-scale problems, CG and its variants are industry standards.

Memory Note: The full conjugate direction method requires storing all n directions $d_1, \dots, d_n$, which costs $O(n)$ memory. The Conjugate Gradient (CG) variant, which we’ll see next, stores only one direction, reducing memory to $O(n)$ total.

Summary and Connections

The Big Picture:

1. **Theorem 1** shows that Q -conjugate directions cannot be redundant—they span the space efficiently.
2. **Theorem 2** derives the exact step size formula that ensures optimal progress along each direction.
3. **Theorem 3** shows that after moving along d_k , the gradient becomes orthogonal (in the Q -sense) to all previous directions—we never need to revisit them.
4. **Theorem 4** formalizes the geometry: each iteration expands the subspace we've optimized over, with no backtracking.
5. **Corollary** concludes: we solve the problem in exactly n steps, guaranteed.

Why This Matters: These theorems answer the central question of optimization: “How do we choose search directions to avoid wasted effort?” The answer is conjugacy with respect to the Hessian. This idea underpins not only conjugate directions but also interior-point methods, second-order optimization, and modern large-scale algorithms.

Key Mathematical Insights:

- **Q -Conjugacy as Generalized Orthogonality:** In the Q -inner product, conjugate directions are orthogonal. This is the foundation for all proofs.
- **Exact Line Search is Essential:** Approximating the step size breaks the orthogonality property and destroys finite termination.
- **Reduced Problem Formulation:** Viewing the subspace optimization as a reduced quadratic problem cleanly justifies the expanding subspace property.
- **Finite Termination in Exact Arithmetic:** For quadratics, convergence is guaranteed in n steps, a stark contrast to gradient-based methods.

Practical Implementation Notes:

- In practice, numerical round-off errors accumulate. After n steps, x_n may not be exactly optimal. Restarting, refinement, or using a tolerance is common.
- For non-quadratic functions, CG is restarted every n iterations or when some condition (e.g., $\|\nabla_k\|^2$ increase) is detected.
- The conjugate direction method requires knowing or constructing the conjugate directions $d_1, \dots, d_n$ in advance. The Conjugate Gradient method (Lecture 15) does this automatically.
- Memory usage: full conjugate direction method requires storing n vectors; CG reduces this to 1 direction (plus a few scalars).
- Computational cost: each iteration requires one matrix-vector product (Qd_k) and a few dot products.

Recommended Homework / Discussion

- **Verification:** Verify Theorem 1 in a 2D example with a non-identity Q . Compute two Q -conjugate directions by hand and check they are linearly independent.
- **Step Size:** Derive the exact step size for the function $f(x) = \frac{1}{2}(x_1^2 + 4x_2^2)$ and a direction $d = (1, 1)^T$. Compare to gradient descent.
- **Simulation:** Implement the conjugate direction method for a small 2×2 or 3×3 problem. Observe exact convergence in n steps.
- **Extension:** Research the Conjugate Gradient method and discuss how it automatically generates Q -conjugate directions using only gradient information.
- **Limitation:** Explain why conjugate direction methods fail on non-convex problems. What goes wrong with the finite-termination guarantee?